

LiteGrip

LiteGrip Software Development Manual

NEXFORM ROBOTICS

Version: V1.2.0 Date: November 5, 2026

Revision History

Version	Date	Revision
V1.2.0	2026-11-05	Revised the hardware list and the physical-layer notes for the shipped configuration; the terminator is built in
V1.1.2	2026-11-03	Removed the in-document contents list and condensed the revision history
V1.1.0	2026-10-22	Restructured the manual for end users and merged the SDK and ecosystem chapters
V1.0.0	2026-09-24	Created the LiteGrip Software Development Manual skeleton

Contents

1 Preface . . . 1
- 1.1 Software components and dependencies . . . 1
- 1.2 Units . . . 2
- 1.3 Safety . . . 3
  - 1.3.1 Overview . . . 3
  - 1.3.2 Safety symbols . . . 3
  - 1.3.3 Safety precautions (software side) . . . 3
  - 1.3.4 Responsibility and standards . . . 4
  - 1.3.5 Risk assessment . . . 4

2 SDK guide . . . 5
- 2.1 Install the SDK . . . 5
- 2.2 Configure the CAN interface . . . 5
- 2.3 Connect 24 V power . . . 5
- 2.4 Load the calibration . . . 6
- 2.5 Minimal example . . . 6
- 2.6 Enable and handle faults . . . 7
- 2.7 Motion control . . . 7
- 2.8 Grasping and force control . . . 8
- 2.9 Read state . . . 9
- 2.10 Hand-guiding . . . 10
- 2.11 Calibration . . . 10
- 2.12 Exceptions and error handling . . . 11

3 Other languages and ecosystem . . . 12
- 3.1 C++ SDK . . . 12
- 3.2 ROS 2 and MoveIt 2 . . . 13
- 3.3 Simulation . . . 14
- 3.4 Host application (LiteGrip Studio) . . . 14
- 3.5 Teleoperation and trajectory replay . . . 15

4 FAQ . . . 15
- 4.1 Q&A . . . 15

5 Appendix 1: CAN protocol . . . 16
- 5.1 Physical layer and frame types . . . 16
- 5.2 MIT control frame . . . 18
- 5.3 Status and error codes . . . 19

5.4 Parameter registers 20
5.5 Communication timing 21

6 Appendix 2: Connection troubleshooting 23
6.1 CAN interface configuration 23
6.2 Connection troubleshooting 23

1 Preface

English | 简体中文

LiteGrip is an adaptive two-finger parallel gripper from NEXFORM ROBOTICS, designed for research and education, AI robot development, and lightweight industrial automation, with an effective stroke of 85.000 mm. The gripper communicates with the host computer through a USB-CAN adapter (classic CAN, 1 Mbps) and ships with a Python SDK, a C++ SDK, a ROS 2 driver, and simulation models.

This manual is for integration engineers. It covers installing and calling the SDK, grasping and calibration, exception handling, the ecosystem components, and a condensed CAN protocol appendix. Product specifications, safety notices, and selection guidance are in the Product Manual; installation, wiring, and the first run are in the Usage Tutorial.

1.1 Software components and dependencies

Component	Repository	Role
Python SDK	litegrip-python	Primary interface, and the subject of this manual
C++ SDK	litegrip-cpp	C++17 interface without ROS
ROS 2 driver	litegrip-ros2	`ros2_control` hardware interface
MoveIt 2 configuration	litegrip-moveit2	Planning and demonstration for the gripper
Model	litegrip-description	URDF and meshes
Simulation	litegrip-pybullet, litegrip-mujoco, litegrip-isaacsim	Model-in-the-loop validation
Host application	litegrip-studio	Graphical console

The Python SDK runs in this environment:

Item	Requirement
Operating system	Linux only, with SocketCAN support in the kernel
Architecture	x86_64 or arm64
Python	3.8 or later

Item	Requirement
Runtime dependencies	None; only the Python standard library and SocketCAN
Optional dependency	`eclipse-zenoh>=1.0`, only for the zenoh teleoperation link
Hardware	USB-CAN adapter, 24 V supply (120 Ω terminator built into the gripper)

The Python SDK and the simulation packages are not published to PyPI; install them from a source checkout.

1.2 Units

Parameter	Unit	Description
Aperture / position	mm	Millimetres, the true distance between the inner faces of the two fingers
Motor angle	rad	Radians
Velocity	mm/s, rad/s	Millimetres per second, radians per second
Torque	N·m	Newton-metres
Gripping force	N	Newtons
Temperature	°C	Degrees Celsius

The SDK position p and “aperture” in this manual are the same quantity: 0 at full closure and 85.000 mm at full opening, with a per-finger stroke of 42.500 mm. It is not a motor angle and not a per-finger displacement; it is exactly what a caliper measures between the two fingertips.

> **Note**: Motor angles and calibrated endpoints differ from unit to unit, so this manual does not print those rad values. Read the calibration file for your unit when you need them, or use the millimetre interfaces directly.

1.3 Safety

1.3.1 Overview

This section states the safety principles you must follow when driving the gripper from software. The gripper grips, collides, and releases when power is lost; users must fully recognize the operating risks and use the SDK only after training.

1.3.2 Safety symbols

This manual uses the severity labels below; the graphical symbols on the product are the authority:

- Danger: a hazardous situation that may cause death, personal injury, or severe equipment damage.
- Warning: a hazardous situation that, if not avoided, could cause death, personal injury, or severe equipment damage.
- High temperature: hot parts, where contact may cause burns or equipment damage.
- Caution: a general warning that, if not avoided, may cause personal injury or equipment damage.

1.3.3 Safety precautions (software side)

1. **Software is not a safety limit.** A position target beyond the calibrated stroke is silently clamped to the endpoint, and out-of-range kp, kd and dq are silently saturated; the caller receives no indication. “Commands must not go out of range” must be implemented by the integrator in the application layer.
2. **You must provide a hardware emergency stop loop whose action does not depend on CAN communication or host software.** If the process hangs, CAN drops, or Python raises, the software side cannot stop the gripper.
3. **You must provide a mechanical stop** as the last line of defense. A calibrated endpoint is not a hard limit.
4. **Do not treat `stop()` or `disable()` as a safety stop.** `stop()` sends one zero-torque frame, leaves the motor enabled and back-drivable; the return value of `disable()` only means the command was sent, not confirmed by feedback.
5. **Do not treat a function’s return value as proof that the motion happened.** A motion call returning means the frames were sent; only a feedback frame proves the actual state.
6. **Do not use any force parameter before force calibration is complete.** `force_n` is converted through a nominal coefficient, is not a calibrated value, and must not be used for safety decisions or load design.
7. **Disconnect the 24 V supply before installation, maintenance, or cleaning.**
8. Debugging and calibration must be supervised by a person on site, with the hardware emergency stop available. On abnormal noise, binding, unusual temperature rise, or a jump in the position reading, stop immediately and cut the 24 V supply.
9. **The driver disables itself if it receives no frame from the host for 0.4 s.** The controller must send frames periodically; with this protection disabled the motor holds the last torque command after a dropout, so do

not disable it without an assessment.

1.3.4 Responsibility and standards

The gripper can form a complete system with other equipment, and this manual does not cover the design, installation, and operation of that complete system. The safety of the complete system depends on how it is integrated; users must carry out a risk assessment under the laws and safety standards of their country or region and take the corresponding protective measures.
Starting to use this gripper means you have read, understood, and accepted all the terms of this manual and of the safety information. Users are responsible for their own actions and their consequences, and must use the gripper only for legitimate purposes. We accept no liability for personal injury, accidents, or property damage caused by violation of the usage requirements or by force majeure.

1.3.5 Risk assessment

From the software side, a risk assessment must consider these failure modes in particular:

Failure mode	Consequence	Mitigation
Host process hangs	Mechanism holds the last command	Driver communication timeout, plus a hardware emergency stop
CAN drops	Software cannot stop the gripper	Driver auto-disables on timeout, plus a hardware emergency stop
Position reading fails	Closed-loop judgment is wrong	Host validates readings, plus a hardware emergency stop
Target calculation is wrong	Mechanism drives into an endpoint	Application-layer limiting, plus a mechanical stop
Several masters transmit at once	Commands overwrite each other	Keep a single master and exclude concurrency in software

Users must judge through the risk assessment whether a hazard is an unacceptable risk and take the corresponding measures. The gripper is not suitable for people who lack the guidance of a qualified professional or who do not have full civil capacity.

2 SDK guide

2.1 Install the SDK

The Python SDK is not published to PyPI, so installation starts from a checkout:

```
git clone https://github.com/nexform-tech/litegrip-python.git
cd litegrip-python
python3 -m pip install .
python3 -c "import litegrip; print(litegrip.__version__)"
```

After a successful install the last command prints the version. Running directly from source without installing prints `0.0.0+source`, which marks "imported from source", not an error.
To run without installing, point PYTHONPATH at the source tree:

```
PYTHONPATH=/path/to/litegrip-python/src python3 your_script.py
```

> **Note**: On the machine that drives the real gripper, use `pip install .`, not `pip install -e .`. An editable install points the interpreter at your working copy, and one `git checkout` there silently changes what the machine actually runs.

2.2 Configure the CAN interface

The interface must exist, be up, and use classic CAN at 1 Mbps. Configuring it needs root or `CAP_NET_ADMIN`.

```
ip -details link show can0
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 fd off
sudo ip link set can0 up
```

> **Note**: `fd off` cannot be omitted. If the interface is opened in CAN FD mode, communication fails completely even with the correct baud rate, and the symptom looks just like "no 24 V connected".

Without hardware, verify the software environment on a virtual bus:

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
```

There is no motor on a virtual bus, so `enable()` exhausts its retries and returns `ok=False`. That is expected behavior.

2.3 Connect 24 V power

The motor needs 24 V to produce torque. Connect it before the first motion command.

> **Note**: CAN communication is powered independently by the USB-CAN adapter, so without 24 V you can still read state and parameters, but the motor reports an undervoltage fault (`ERR = 0x9`) and does not respond to motion commands. When "the command was sent but nothing moves", check 24 V first.

2.4 Load the calibration

The travel endpoints differ from unit to unit, so the calibration must be loaded before motion:

```
gripper.connect()
gripper.load_calibration()   # the factory calibration built into the SDK
gripper.enable()
```

> **Note**: `connect()` does not load the calibration. Without one, `open()`, `close()`, `grasp()` and every positioning method raise `CommandError` rather than running on placeholder coefficients.

Pass `path=` to `load_calibration()` to read a file for your unit, or `template="normal"` / `template="reverse"` to declare the mounting direction. The two cannot be passed together.

2.5 Minimal example

```
from litegrip import LiteGrip

with LiteGrip(channel="can0", can_id=0x08) as gripper:
    gripper.load_calibration()
    gripper.enable()

    gripper.open()                                      # 50 mm/s; the shipped calibration stops at 80 mm
    gripper.close()                                     # presses onto the closed-side endpoint
    result = gripper.grasp(force_n=20.0, hold_s=3.0)    # close onto the object, then hold 20 N
    print(result.ok, result.reached, result.stalled, result.cycles)
```

Run `python3 first_move.py`. With an object grasped the typical output is `True False True 12`:

Field	Meaning
ok	The hold phase finished normally; this is the field to read
`reached`	The target angle was reached without a stall, that is a free close
`stalled`	The position stopped changing, which is the result when an object is grasped

Field	Meaning
`cycles`	How many hold cycles ran

Note: A successful powered grasp reports `stalled=True` and `reached=False`; do not expect `reached=True`. Grasping an object and closing on the endpoint look identical to the stall test, so read `get_position()` to tell them apart: a free close stops at the closed endpoint, a grasp stops before it.

2.6 Enable and handle faults

```
gripper.enable()        # enable and wait for feedback confirmation
gripper.disable()       # disable, torque to zero
gripper.clear_fault()   # clear a driver fault
gripper.stop()          # send one zero-torque frame
```

`enable()` clears any latched fault, enables the motor, then waits for one feedback frame confirming `ERR == 0x1`. When its retries are exhausted it returns `EnableResult(ok=False)` without raising, so check `ok`.

Method	Description
`enable(retries=None)`	Enable and confirm; `retries=None` takes `MotionConfig.enable_retries` (3)
`disable()`	Disable; the return value only means the command was sent
`clear_fault()`	Disable, clear the fault, enable again, and confirm
`stop()`	Stop with one zero-torque frame; neither enables nor disables, and latches nothing

Note: `stop()` is not a safety-rated stop. It sends one frame and returns; the motor stays enabled and accepts the next command. A real emergency stop cuts the 24 V supply with a hardware switch.

2.7 Motion control

Method	Effect
`open()`	Opens to `work_stroke_mm` or the open endpoint; the shipped calibration stops at 80 mm
`close()`	Presses onto the closed-side endpoint; takes no force argument
`grasp()`	Closes to a stall, then holds a force; see the next section
`goto(position_mm)`	Moves to an absolute position (mm)
`goto_rad(position_rad)`	Moves to an absolute position (rad)
`move_to(target_rad)`	A longer-duration version of `goto_rad()`
`move_at_speed(target_mm, speed_mm_s=30.0)`	Moves at a given linear speed
`move_at_speed_rad(target_rad, speed_rad_s=0.5)`	Moves at a given angular speed
`home()`	Returns to the calibrated closed position

```
with LiteGrip(channel="can0", can_id=0x08) as gripper:
    gripper.load_calibration()
    gripper.enable()
    gripper.open()
    gripper.goto(60.0)
    gripper.close()
```

> **Note**: A target beyond the calibrated endpoints is silently clamped, with no exception and no indication. To reject out-of-range targets, compare against `config.pos_closed_rad` and `config.pos_open_rad` before the call.
>
> **Note**: `duration` only sets how long the frame sequence lasts; it is not a speed limit. To limit speed, use `move_at_speed()` or `move_at_speed_rad()`.

2.8 Grasping and force control

```
grasp(force_n=None, hold_s=0.0, *, progress=None) -> GraspResult
```

`force_n=None` takes `MotionConfig.force_n` (20.0 N). `hold_s=0.0` holds until a fault, a travel-guard trigger, or Ctrl+C.
`grasp()` first closes toward the object; once it detects that the position has stopped changing it enters the

hold phase. A hold frame carries no position or velocity gain, so its only output is the `force_n × 0.1` N·m feed-forward torque.

`GraspResult` field	Meaning
ok	The hold phase finished normally
`reached`	The target angle was reached without a stall, that is a free close
`stalled`	The position stopped changing, the normal result when an object is grasped
`cycles`	How many hold cycles ran; 0 means the hold phase was never entered
`travel_guard`	The runaway guard triggered and removed the torque
`state`	The `GripperState` at the end

> **Note**: `force_n` has never been force-calibrated. `0.1 N·m/N` is a nominal coefficient, and `force_n=0` means zero feed-forward torque, not a position hold. Until force calibration is complete, do not use this parameter for safety decisions or load design.
>
> **Note**: There is no "apply a force at the current position" interface, deliberately. A hold frame carries no position error, so with nothing between the fingers the torque is the only force and the mechanism accelerates. Use `grasp()` when you need force.

2.9 Read state

```
state = gripper.get_state(wait=True)
print(state.position_mm, state.temperature_mos, hex(state.error_code))
```

Field	Description
`position_mm`	Actual aperture (mm)
`position_rad`	Motor angle (rad)
`velocity_rad_s`	Velocity
`torque_nm`	Torque
`force_n`	Estimated gripping force, uncalibrated

Field	Description
`temperature_mos` / `temperature_coil`	MOS temperature / coil temperature
`error_code`	Error code; 0 means disabled, 1 means enabled
`fresh` / `data_age_s`	Whether this read was answered by a frame / how old that frame is

`wait=True` waits up to 50 ms for a new feedback frame; `wait=False` returns the cache directly, which suits a high-rate control loop.

> **Note**: Feedback is polled; the motor answers only after it receives a frame from the host, and passively listening on the bus never yields a feedback frame. An enabled but idle gripper sends nothing, so `get_state(wait=True)` waits the full 50 ms and returns the previous frame.
>
> **Note**: `force_n` is derived as `torque_nm × 10`, and 10.0 is a nominal coefficient, not a calibrated value. `aperture_mm` is already the full aperture and equals `position_mm`; do not multiply it by 2.

2.10 Hand-guiding

```
gripper.enter_zero_gravity()       # the gripper goes soft and can be pushed by hand
input("Press Enter once the position is set...")
gripper.exit_zero_gravity()        # hold the current position
```

`enter_zero_gravity()` keeps the motor enabled but continuously sends frames with `kp=0, kd=0, tau=0`, so the mechanism can be pushed freely.

> **Note**: Zero gravity mode does not hold position; gravity and friction decide where the fingers stop after you let go. To "hold where you let go", you must call `exit_zero_gravity()`. Leaving the mode performs no range check; a position pushed beyond the stroke is held directly.
>
> **Note**: `enter_zero_gravity(duration=...)` only stops sending frames when the timer ends; it does not return to position holding on its own. The controller must keep sending frames, or the driver disables itself after 0.4 s.

2.11 Calibration

Method	Drive	How limits are detected	Needs a person
`zero()`	Motor	Stall detection, both ends in one call	On site

Method	Drive	How limits are detected	Needs a person
`calibrate()`	Motor	Stall detection	On site
`calibrate_guided()`	Motor	Enter at each endpoint, or automatic stall detection	Confirm each endpoint
`calibrate_manual()`	Hand push (zero gravity)	Push to each end and record the extremes	Push throughout

`zero()`, `calibrate()` and `calibrate_guided()` drive all the way into the mechanical hard stops and do not stop on their own. Before running them, confirm the structure can take the force and that someone is present with a working hardware emergency stop. `calibrate_manual()` is recommended; it applies no motor force to the mechanism.
Save the result with `save_calibration()`:

Method	Description
`save_calibration(path=None)`	Writes to `~/.litegrip/<channel>_calibration.json` by default and returns the path
`load_calibration(path=None, template=None)`	Loads a calibration into `config`
`load_template("normal")` / `load_template("reverse")`	Loads a mounting-direction template

> **Note**: `load_calibration()` does not validate the contents; it only handles a missing file and a JSON parse failure. Recalibrate after replacing the gripper, motor, or fingers, and after a collision or overload.

2.12 Exceptions and error handling

Exception	Trigger
`LiteGripError`	Base class, optionally carrying an `error_code`
`CommError`	A CAN bus read or write failed

Exception	Trigger
`ConnectError`	The CAN interface is unavailable, or the motor does not answer
`CommandError`	Not calibrated, zero stroke, a command frame that cannot be sent, or an invalid argument combination
`CANTimeoutError`	No response on the bus
`HardwareError`	The driver reports undervoltage, overcurrent, overtemperature, or another fault
`NotInitializedError`	A method that needs a connection or enable was called without one

```
from litegrip import LiteGrip, HardwareError, CommandError, LiteGripError

try:
    with LiteGrip(channel="can0", can_id=0x08) as gripper:
        gripper.load_calibration()
        gripper.enable()
        gripper.grasp(force_n=20.0, hold_s=3.0)
except CommandError:
    print("calibration missing or command rejected")
except HardwareError as error:
    print(f"driver fault: {error}")
except LiteGripError as error:
    print(f"SDK error: {error}")
```

> **Note**: Clamping and saturation raise nothing. An out-of-range target is silently clamped, out-of-range kp, kd and dq are silently saturated, and `force_n` is not range-checked. Non-finite values and a `tau` beyond the model range raise `CommandError`. A program that only catches exceptions cannot see clamping.

3 Other languages and ecosystem

3.1 C++ SDK

litegrip-cpp is a C++ SDK independent of ROS: C++17, CMake 3.16 or later, Linux and SocketCAN, with no third-party dependency beyond the standard library, `pthread`, and the SocketCAN headers.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

```
ctest --test-dir build --output-on-failure
cmake --install build --prefix "$HOME/.local"
```

The tests need no hardware, no CAN interface, and no root. To use it in your own project:

```
find_package(litegrip_cpp REQUIRED)
target_link_libraries(my_app PRIVATE litegrip_cpp::litegrip_cpp)
```

> **Note**: A program built outside the install prefix needs `LITEGRIP_DATA_DIR="$HOME/.local/share/litegrip_cpp"` to find the calibration data; without it `load_calibration()` prints the paths it tried and returns `false`.

The C++ interface broadly matches Python, with a few differences:

- Trajectory recording and replay, teleoperation, and `write_zero()` exist only in Python.
- Neither SDK has `set_force()`; use `grasp()` when you need force.
- `close()` takes a speed and a progress callback, not a force.
- The command velocity is capped by `kMaxCommandVelocityCeilingRadS` (1.5 rad/s) and can only be lowered.

3.2 ROS 2 and MoveIt 2

litegrip-ros2 provides a `ros2_control` `SystemInterface` plugin backed directly by the C++ SDK, with no Python daemon in the path. Motion is carried by one joint, `gripper_opening_joint`, and the two fingers are its mimics.

Parameter	Default	Description
`channel` / `can_id`	`can0` / `0x08`	Bus interface and motor ID
`dry_run`	`true`	Sends no motion frames while true
`hardware_enable`	`false`	Must be true to drive the motor
`max_velocity_rad_s`	`1.5`	Command velocity cap, lowerable only
`max_feedback_velocity_rad_s`	`-1.0`	Worst-case feedback velocity bound; `-1.0` means unspecified and refuses motion
`torque_limit_nm`	`3.5`	Total control torque budget

Motion frames are sent only when `dry_run=false` and `hardware_enable=true`. `max_feedback_velocity_`

`rad_s` must be calibrated on your unit; `-1.0` is deliberate fail-safe behavior.

```
ros2 launch litegrip_moveit_config demo.launch.py
ros2 launch litegrip_moveit_config demo.launch.py \
  dry_run:=false hardware_enable:=true \
  max_feedback_velocity_rad_s:=0.9
```

The model lives in litegrip-description; view it with `ros2 launch litegrip_description display.launch.py`.

> **Note**: `litegrip-ros1` is an empty repository with no source, packages, or release. Do not plan ROS 1 integration around it.

3.3 Simulation

Package	Entry point	What it provides
litegrip-pybullet	`python3 examples/01_hello_sim.py --headless`	`GripperSim`, plus sim→real and real→sim examples
litegrip-mujoco	`from litegrip_mujoco import MujocoGripper`	`MujocoGripper`, `MirrorMode`, `DualGripper`
litegrip-isaacsim	`ISAAC_SIM_PATH=... ./run_gripper.sh`	An Isaac Sim node, and a bridge to the real CAN bus

Both Python simulation environments use the same model from `litegrip-description`. None of the three is published to PyPI; install from a source checkout.

3.4 Host application (LiteGrip Studio)

litegrip-studio is a PyQt5 console for driving, calibrating, and monitoring the gripper. It needs Python 3.10 or later, plus `PyQt5`, `pyqtgraph`, and `numpy`.

```
./run_litegrip_studio.sh sim        # the built-in simulated gripper
./run_litegrip_studio.sh gui        # a real gripper on can0
./run_litegrip_studio.sh selftest   # self-check, no Qt and no hardware
./run_litegrip_studio.sh test       # the test suite
```

The SDK is bundled under `src/litegrip`, so a clone runs without extra installation. Anything you need to script still goes through the SDK.

3.5 Teleoperation and trajectory replay

The Python SDK provides `GripperTeleop` for leader-follower teleoperation; the wire carries an openness normalized to `[0, 1]`, so the two ends need not share a calibration. Record with `TrajectoryRecorder` and replay with `TrajectoryPlayer`. The example scripts are `examples/teleop.py` (one process per end) and `examples/trajectory.py` (record, list, and replay).

> **Note**: You can subclass `TeleopTransport` and implement `pub`, `sub`, and `close` to use your own link; do not run teleoperation and trajectory replay on the same gripper at the same time.

4 FAQ

4.1 Q&A

Q: Which platforms does the SDK support? A: Linux only, with SocketCAN support in the kernel. The SDK uses only the Python standard library and has no runtime dependencies.
Q: Does `pip install litegrip` work? A: No. The package is not published to PyPI; clone the repository and run `python3 -m pip install .`.
Q: Why is `fd off` required when configuring CAN? A: This product uses classic CAN. A CAN FD configuration fails to communicate at all, and the symptom looks like "no 24 V connected".
Q: Why do I see nothing when I listen on the bus? A: Feedback is polled; the motor answers only after it receives a frame from the host, so you must send one first.
Q: What do I do after connecting? A: Call `load_calibration()`, then `enable()`. `connect()` does not load the calibration by itself.
Q: `open()` raises `CommandError` saying the configuration is not calibrated. What is wrong? A: The calibration was not loaded. Call `load_calibration()`, or run `zero()`.
Q: `goto()` goes to the wrong place without raising. Why? A: A target beyond the calibrated endpoints is silently clamped; that is by design. To reject out-of-range targets, compare against the endpoints first.
Q: A function returned `True`. Does that mean it worked? A: It depends on the function. `enable()` returning true is backed by a feedback frame; `disable()` returning true only means the command was sent; a motion call returning means the frames were sent, and you must read `get_state()` to know whether the mechanism arrived.
Q: Why does `close()` not stop at the force I set? A: `close()` has no force parameter. Only `grasp()` takes `force_n`, and that parameter has never been force-calibrated.
Q: `grasp()` returned `stalled=True`. Did it grasp something? A: Not necessarily. A free close onto the endpoint also stalls. Read `get_position()`: a free close stops at the closed endpoint, a grasp stops before it.
Q: `grasp()` returned `ok=False`. Is it broken? A: Not necessarily. `ok` only reports whether the hold phase finished normally; read it together with `reached`, `stalled`, and `cycles`.
Q: The fingers stopped outside the stroke. What now? A: Bring them back with `goto_rad()`, which clamps

the target to the calibrated endpoints. If the mechanism is jammed, clear the interference by hand rather than applying more force.

Q: If I let go in zero gravity mode, will the fingers drop? A: They will move. Zero gravity mode applies no torque; call `exit_zero_gravity()` to hold the position.

Q: The position reading does not match a caliper measurement. A: A fixed percentage difference means the calibration scale is wrong; a fixed offset means you measured from a different reference face. The reading is the aperture between the two fingertips.

Q: Can several hosts control the gripper at once? A: No. Keep a single CAN master and exclude concurrency in software. To drive several grippers from one process, use `litegrip.can.controller`.

Q: What is the technical support channel? A: The NEXFORM ROBOTICS technical team. Contact details are in the documents shipped with the product and through the sales channel.

5 Appendix 1: CAN protocol

This appendix is for integration engineers who do not use the SDK, or who implement control in another language. The SDK's clamping and zero-torque behavior do not apply automatically: range checks, speed limits, the emergency-stop loop, and periodic frame transmission are yours to implement.

5.1 Physical layer and frame types

Item	Value
Bus type	Classic CAN (CAN 2.0)
Baud rate	1 Mbps
CAN FD	Not used; must be disabled
Data frame length	8 bytes
Terminating resistor	120 Ω built in; no external terminator needed

Name	Value	Meaning
Motor CAN ID (`ESC_ID`)	`0x08`	Control and command frames from host to motor
Feedback ID (`MST_ID`)	`0x18`	Frames from motor to host

Name	Value	Meaning
Broadcast ID	`0x7FF`	Status-refresh and parameter frames, with the target motor ID in the data

Frame type	Direction	Arbitration ID	Purpose
MIT control frame	Host → motor	`0x08`	Position, velocity, stiffness, damping, feed-forward torque
Command frame	Host → motor	`0x08`	Enable, disable, clear fault, set zero
Status-refresh frame	Host → motor	`0x7FF`	Requests one status frame without changing the output
Parameter frame	Host → motor	`0x7FF`	Reads and writes registers, saves to Flash
Feedback frame	Motor → host	`0x18`	Status, position, velocity, torque, temperature, error code

A command frame has bytes 0 to 6 all set to `0xFF` and the command code in byte 7:

Command	Code	Full data bytes
Enable	`0xFC`	FF FF FF FF FF FF FF FC
Disable	`0xFD`	FF FF FF FF FF FF FF FD
Clear fault	`0xFB`	FF FF FF FF FF FF FF FB
Set zero	`0xFE`	FF FF FF FF FF FF FF FE

Command	Code	Full data bytes

A status-refresh frame goes to `0x7FF` with the data `[can_id low byte, can_id high byte, 0xCC, 0x00]`; for motor CAN ID `0x08` that is `08 00 CC 00`.

> **Note**: Only status-refresh and parameter frames are addressed with the broadcast ID `0x7FF`. MIT control and command frames are addressed by arbitration ID and must go to `0x08`. At the wrong address no motor answers or acts, and the symptom looks just like “no 24 V connected”.
>
> **Note**: “Set zero” records the current position as the mechanical zero and cannot be undone. The gripper’s zero comes from calibration; do not send this command.

5.2 MIT control frame

Field	Width	Range	Description
Position q	16 bits	±12.5 rad	Target angle
Velocity dq	12 bits	±30 rad/s	Target velocity
Position stiffness kp	12 bits	0 ~ 500	Higher is stiffer
Damping kd	12 bits	0 ~ 5	Damping coefficient
Feed-forward torque `tau`	12 bits	±10 N·m	Feed-forward torque

The torque output inside the motor is:

```
tau_out = kp * (q_target - q_actual) + kd * (dq_target - dq_actual) + tau
```

Byte layout:

Byte	Content
0	`q[15:8]`
1	`q[7:0]`
2	`dq[11:4]`
3	`dq[3:0]` in the high 4 bits, `kp[11:8]` in the low 4 bits

Byte	Content
4	`kp[7:0]`
5	`kd[11:4]`
6	`kd[3:0]` in the high 4 bits, `tau[11:8]` in the low 4 bits
7	`tau[7:0]`

Quantization truncates rather than rounds:

```
code = int((value - min) * (2^bits - 1) / (max - min))
value = code * (max - min) / (2^bits - 1) + min
```

The decoded value is therefore never greater than the commanded value; the error falls in `(-1 LSB, 0]`.

> **Note**: A zero-torque frame still carries the q field, but `kp = kd = 0` makes the position and velocity terms identically zero, so q produces no output whatever it holds. The SDK fills q with the current measured position in a zero-torque frame.
>
> **Note**: Out-of-range kp and kd are silently saturated to the encoding range; non-finite values and a `tau` beyond the model range are rejected.

5.3 Status and error codes

A feedback frame uses arbitration ID `0x18`:

Byte	Content
0	`ERR[7:4]` in the high 4 bits, `can_id[3:0]` in the low 4 bits
1	`q[15:8]`
2	`q[7:0]`
3	`dq[11:4]`
4	`dq[3:0]` in the high 4 bits, `tau[11:8]` in the low 4 bits
5	`tau[7:0]`
6	MOS temperature (°C)
7	Coil temperature (°C)

q, dq, and `tau` decode with the same rules and ranges as the MIT control frame.

Code	Meaning	Category
`0x0`	Disabled	Status
`0x1`	Enabled	Status
`0x8`	Overvoltage fault	Fault
`0x9`	Undervoltage fault	Fault
`0xA`	Overcurrent fault	Fault
`0xB`	MOS overtemperature fault	Fault
`0xC`	Coil overtemperature fault	Fault
`0xD`	Communication loss	Fault
`0xE`	Overload	Fault

Note: `0x8` and `0x9` are easy to read backwards; the order is 8 overvoltage, 9 undervoltage.

Note: These are the common codes only. The driver also reports faults outside this table; the SDK shows “unknown error” for codes it does not recognize, so consult the driver indicator when needed.

Note: Judge the enable state from a feedback frame. Only `ERR = 0x1` means enabled; at `ERR = 0x0` the motor accepts position commands but produces no torque, which looks like “the command was sent but nothing moves”.

5.4 Parameter registers

Parameter frames always go to `0x7FF`:

Operation	Data bytes
Read	`[can_id low, can_id high, 0x33, RID, 0, 0, 0, 0]`
Write	`[can_id low, can_id high, 0x55, RID, d0, d1, d2, d3]`
Save	`[can_id low, can_id high, 0xAA, 0x01, 0, 0, 0, 0]`

A response has the same structure: byte 2 echoes the operation code, byte 3 is the RID, and bytes 4 to 7 are the data. Integer registers are little-endian unsigned 32-bit; float registers are little-endian IEEE 754 single precision.

RID	Name	Type	Description
0	UV_Value	Float	Undervoltage protection threshold (V)
2	OT_Value	Float	Overtemperature protection threshold (°C)
3	OC_Value	Float	Overcurrent protection threshold (ratio, not amperes)
6	MAX_SPD	Float	Maximum speed, the real speed limit
7	MST_ID	Integer	Feedback frame ID
8	ESC_ID	Integer	Motor CAN ID
9	TIMEOUT	Integer	Communication timeout, in 50 µs units
10	CTRL_MODE	Integer	Control mode
14	sw_ver	Integer	Firmware version
21	PMAX	Float	Position mapping range (rad)
22	VMAX	Float	Velocity mapping range (rad/s)
23	TMAX	Float	Torque mapping range (N·m)
29	OV_Value	Float	Overvoltage protection threshold (V)

Examples: reading TIMEOUT (RID 9) sends `08 00 33 09 00 00 00 00`; writing TIMEOUT as 8000 (0.4 s) sends `08 00 55 09 40 1F 00 00`; saving parameters sends `08 00 AA 01 00 00 00 00`.

Note: OC_Value is a ratio; `0.8` means 80%, not 0.8 A.

Note: PMAX, VMAX, and TMAX are the mapping ranges of the MIT frame fields, not protection thresholds. The real speed protection is MAX_SPD.

Note: Written parameters do not survive a power cycle unless you send the save frame explicitly, and the motor must be disabled before saving.

5.5 Communication timing

Item	Value	Description
Enable success criterion	`ERR == 0x1`	The status frame must arrive after the command
Enable retries	3	When all fail, `enable()` returns `ok=False`

Item	Value	Description
Disable verification	None	Returns once the disable frame is sent
Status-refresh wait	0.05 s	How long `get_state(wait=True)` waits
Parameter read wait	0.5 s	The default `read_param()` timeout
Control frame period	5 ms (200 Hz)	The cadence during motion
Driver communication timeout	0.4 s	The driver disables itself after this

Step	Direction	Frame
1	Host → motor	Clear fault `FF FF FF FF FF FF FF FB`
2	Host → motor	Enable `FF FF FF FF FF FF FF FC`
3	Motor → host	Feedback frame, check `ERR = 0x1`
4	Host → motor	MIT control frames, sent continuously
5	Motor → host	One feedback frame per control frame received
6	Host → motor	Zero-torque frame
7	Host → motor	Disable `FF FF FF FF FF FF FF FD`
8	Motor → host	Feedback frame, check `ERR = 0x0`

Note: Steps 3 and 8 cannot be omitted. Command frames are one-way; only a feedback frame proves that a command took effect.

Note: The driver disables itself if it receives no frame from the host for 0.4 s, so the controller must keep transmitting periodically.

6 Appendix 2: Connection troubleshooting

6.1 CAN interface configuration

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 fd off
sudo ip link set can0 up
ip -details link show can0
```

Without hardware, verify the software environment on a virtual bus:

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
```

A bus self-check without the SDK:

```
sudo apt install can-utils

# 1) Passive listening: with no frame sent, nothing should appear
candump can0

# 2) In another terminal, send one status refresh
cansend can0 7FF#0800CC00

# 3) Read the firmware version (RID 14)
cansend can0 7FF#0800330E00000000
```

> **Note**: A long silence from `candump` does not mean the gripper is broken. Feedback is polled; send the frame from step 2 before judging.

The safest enable self-check connects only USB-CAN and leaves 24 V off, then sends the enable command five times:

```
for i in $(seq 5); do cansend can0 08#FFFFFFFFFFFFFFFC; sleep 0.01; done
```

The expected feedback has ERR = 0x9 (undervoltage). Receiving 0x9 proves the bus works and the motor answers, which isolates the problem to the supply side.

> **Note**: Once 24 V is connected, sending the enable frame makes the mechanism move for real. Confirm that no person and no obstruction is in the fingers' range before running it.

6.2 Connection troubleshooting

Symptom	Possible cause	Remedy
No feedback frames at all	Listening without sending	Feedback is polled; send a frame first
	Command frames sent to `0x7FF`	Command and MIT frames must go to `0x08`
	CAN FD not disabled	Reconfigure with `fd off`
	Extra terminating resistor added externally	Remove the external terminator; the gripper has 120 Ω built in
Enable has no effect, ERR stays `0x0`	24 V not connected	Connect 24 V
	Using a stale feedback frame	Judge only from a frame that arrives after the command
Feedback `ERR = 0x9`	24 V not connected or too low	Check the supply
No motion after a command, `ERR = 0x1`	Target position equals the current position	Confirm the target value
	`kp = 0`	Check the stiffness parameter
The gripper suddenly disables during motion	Driver communication timeout	Keep transmitting periodically
	Overtemperature or overcurrent protection tripped	Stop and let it cool, then check the load
Feedback `tau` is slightly below the command	Truncating quantization	Normal behavior
Several devices interfere with each other	Multiple masters	Keep a single master and exclude concurrency in software

If you cannot resolve the problem yourself, record the product model and serial number, SDK version, Python version, operating system and kernel version, CAN interface configuration, and the full exception text and stack trace, then contact us.

End of document If you have questions or need more information, contact the NEXFORM ROBOTICS technical team.